

Documentacion Figuritas.app

1. Backend.....	3
1.1 Dependencias.....	3
Frameworks.....	3
Spring-boot.....	3
Spring-Data MongoDB.....	3
Librerías.....	3
Lombok.....	3
Flapdoodle.....	4
JUnit 5 + Mockito.....	4
Herramientas y otros.....	4
Json Web Token.....	4
JaCoCo + SpotBugs.....	4
1.2 Dominio.....	5
1.3 API.....	6
Usuario y Sesión.....	6
POST /usuarios.....	6
POST /administradores.....	6
GET /administrador/estadísticas.....	6
POST /login.....	7
GET /yo.....	7
DELETE /sesión.....	7
Perfil.....	7
POST /perfil/calificaciones.....	7
GET /perfil/calificaciones.....	8
GET /perfil/sugerencias.....	8
GET /perfil/contadores.....	9
GET /perfil/notificaciones.....	9
GET /perfil.....	9
Colecciones.....	9
POST /colecciones/faltantes.....	9
POST /colecciones/repetidas.....	10
GET /colecciones/faltantes.....	10
GET /colecciones/repetidas.....	10
Figuritas.....	11
GET /figuritas.....	11
Propuestas.....	12
POST /propuestas.....	12
PATCH /propuestas/{id_propuesta}/aceptar.....	12
PATCH /propuestas/{id_propuesta}/rechazar.....	12
PATCH /propuestas/{id_propuesta}/cancelar.....	12
GET /propuestas.....	12

Subastas.....	13
POST /subastas.....	13
POST /subastas/{sub_id}/ofertas.....	13
PATCH /subastas/{sub_id}/ofertas/{oferta_id}.....	14
PATCH /subastas/{sub_id}/ofertas/{oferta_id}/cancelar.....	14
PATCH /subastas/{sub_id}/ofertas/{oferta_id}/seleccionar.....	14
PATCH /subastas/{sub_id}/ofertas/{oferta_id}/rechazar.....	14
PATCH /subastas/{sub_id}/cancelar.....	15
PATCH /subastas/{sub_id}/cerrar.....	15
GET /subastas.....	15
1.4 Persistencia.....	16
Paginación.....	16
Objetos Embebidos.....	16
Listas Lazy.....	16
Otras consideraciones.....	17
2. Frontend.....	17
2.1 Dependencias.....	17
Frameworks.....	17
React 19.....	17
Librerías.....	17
React Router 7.....	17
Axios.....	17
Bootstrap.....	18
Herramientas y otros.....	18
Vite 8.....	18
Nginx.....	18
3. Infraestructura.....	18
3.1 Docker.....	18
3.2 Backend.....	18
3.2 Frontend.....	19
4. Uso de la Inteligencia Artificial.....	20
4.1 Creación de test según casos de uso.....	20
4.2 Diseño de las plantillas para el frontend.....	20
4.3 Generación de Componentes React.....	20
4.4 Generación de Documentación.....	21
4.5 Debugging errores del Front y Back.....	21
4.6 Revisión de Pull Request en Github.....	21

1. Backend

1.1 Dependencias

Frameworks

Spring-boot

Esta dependencia nos brinda un framework de trabajo que el equipo conoce en un nivel bastante alto en comparación con otros.

El framework ofrece automatizar la mayoría de controles HTTP internos, y a su vez, una cantidad de dependencias fácilmente integrables con él, permitiendo una flexibilidad atractiva.

Spring-Data MongoDB

Esta dependencia forma parte del framework Spring Boot, nos permite hacer un mapeo del mundo de Objetos a Documental (y viceversa) empleando tags y consultas a nivel de objetos. Tiene la ventaja de su simplicidad de uso, abstrayendo al desarrollador de aspectos como Schema y mapeo de objetos.

Librerías

Lombok

Lombok es una librería que nos permite **enriquecer de métodos** en las clases sin llenar de código las mismas, permite crear getters, setters, constructores, builder, etc.

Los más usados en el proyecto son:

- **@Getters**
- **@Setters**
- **@Builder**: Muchas clases comenzaron a tener muchos constructores básicamente para desarrollar en los test, los cuales no son empleados en condiciones normales. El builder nos permite no llenar de distintos constructores en las clases.
- **@NoArgsConstructor**: Hace constructores sin parametros. Usados para el framework de **Spring Data MongoDB**.
- **@AllArgsConstructor**: Hace constructores con todos los parámetros, usados para test cuando se requiere.
- **@RequiredArgsConstructor**: Hace constructores con aquellos atributos con **@NonNull** o **final**. Son usados en **Controladores, Servicios y Repositorios** para el framework **Spring Boot**.

Flapdoodle

Flapdoodle es una librería que simula una base de datos dentro de la memoria pensada para **entornos de testing**. Con esta herramienta podemos comprobar el correcto

funcionamiento de los repositorios y posibles conflictos que al mockear podríamos no detectar.

Esta librería, junto con el framework **Spring Data MongoDB**, nos permitió desarrollar test de integración empleando una clase abstracta **MongoTestBase**.

JUnit 5 + Mockito

Esta librería nos permite desarrollar test de alta fidelidad y permitiendo mocks de ciertos elementos. Se eligió por el fuerte vínculo que presenta con Spring-boot y facilidad de implementación, haciendo que el trabajo de desarrollar test sea rápido y eficiente.

Herramientas y otros

Json Web Token

Herramienta empleada para mantener una sesión a lo largo del uso del sistema por un usuario. Permite guardar contenidos a elección encriptados y firmados con un secreto, garantizando que cierta información persista en más de un pedido HTTP.

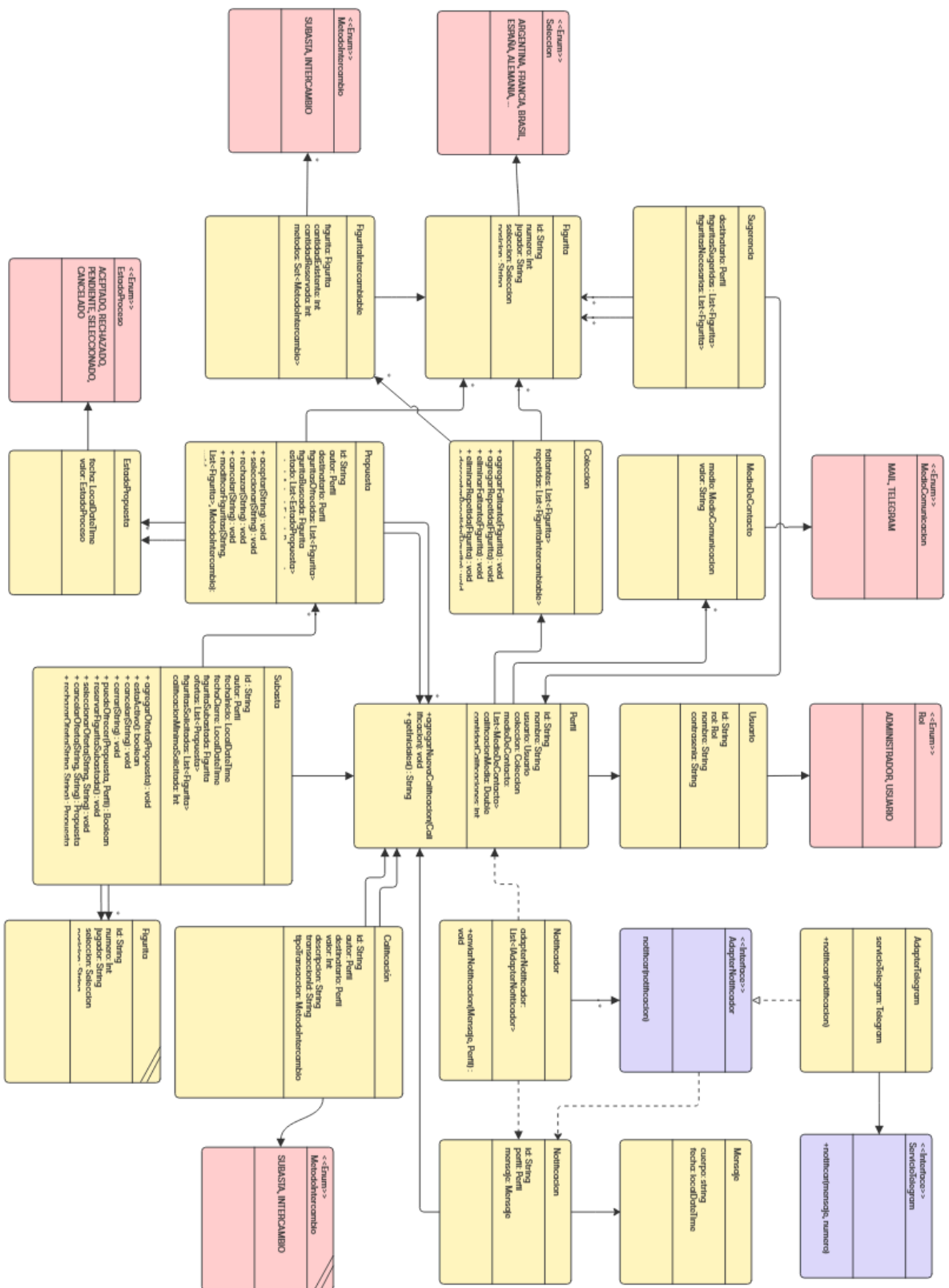
Esta herramienta fue seleccionada debido a su amplio conocimiento en los miembros del equipo, además de permitir tener un control amplio de cada sesión y sus campos.

JaCoCo + SpotBugs

JaCoCo nos brinda el servicio de cobertura de los archivos sumado con SpotBugs. En conjunto nos permite una robustez garantizada del porcentaje que el equipo decidió como válido (80% de paquetes).

JaCoCo nos da una facilidad a la hora de comprobar qué archivos requieren de mayor cobertura en los test, y en consecuencia, aumentar la robustez del sistema en general.

1.2 Dominio



1.3 API

Usuario y Sesión

POST /usuarios

- Respuesta esperada: 204 "No Content".
- Crea y persiste un usuario con un perfil limpio para su posterior login.
- Los roles posibles de usuarios son: "USUARIO" (nivel 0).
- Errores posibles: 400 "Bad Request".
- Se requieren los campos:
 - Body:
 - Nombre (string variable)
 - Contraseña (string variable)
 - Rol ("USUARIO")

POST /administradores

- Respuesta esperada: 204 "No Content".
- Crea y persiste un usuario con un perfil limpio para su posterior login, requiere de la sesión de un administrador de la aplicación.
- Los roles posibles de usuarios son: "USUARIO" (nivel 0) y "ADMINISTRADOR" (nivel 1).
- Errores posibles: 400 "Bad Request", 401 "Unauthorized".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Body:
 - Nombre (string variable)
 - Contraseña (string variable)
 - Rol ("USUARIO" | "ADMINISTRADOR")

GET /administrador/estadísticas

- Respuesta esperada: 200 "Ok".
- Obtiene estadísticas de varios aspectos de la aplicación, devolviendo:
 - Total de usuarios.
 - Total de figuritas publicadas.
 - Total de propuestas.
 - Total de subastas activas
 - De cada estado de propuesta:
 - cantidad de pendientes.
 - cantidad de aceptadas.
 - cantidad de rechazadas.
 - De cada modalidad de figuritas:
 - cantidad de solo intercambio.
 - cantidad de solo subasta.
 - cantidad de intercambio y subasta.
 - De cada selección:
 - Cantidad de intercambios.

- Se requieren los campos:
 - Token (JsonWebToken)
- Errores posibles: 401 "Unauthorized".
- No se solicitan campos.

POST /login

- Respuesta esperada: 204 "No Content".
- Crea una sesión empleando JsonWebToken, agregándole en cookies de HTTP para su posterior uso en validaciones y facilidades de búsqueda.
- Errores posibles: 400 "Bad Request", 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Body:
 - Nombre (string variable)
 - Contraseña (string variable)

GET /yo

- Respuesta esperada: 200 "Ok".
- Busca y devuelve información del usuario logueado empleando el token de la cookie.
- Errores posibles: 401 "Unauthorized".
- Se requieren los campos:
 - Token (JsonWebToken)
- Devuelve:
 - Id de usuario
 - Id de perfil
 - Id de colección
 - rol de usuario logueado

DELETE /sesión

- Respuesta esperada: 204 "No Content".
- Elimina la sesión del servidor, invalidando el token.
- Errores posibles: 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)

Perfil

POST /perfil/calificaciones

- Respuesta esperada: 200 "Ok".
- Persiste una calificación a un usuario destino.
- Errores posibles: 400 "Bad Request", 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Body:

- Valor de puntuación
- Id del usuario calificado
- Descripción de la calificación
- Id de la transacción
- Tipo de transacción

GET /perfil/calificaciones

- Respuesta esperada: 200 "Ok".
- Busca las calificaciones del perfil en sesión dentro del Token.
- Errores posibles: 400 "Bad Request", 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Página
 - Límite por página
- Devuelve:
 - Lista de contenido:
 - Id de calificación
 - Id del perfil autor de la calificación
 - Valor enviada en esa calificación
 - Descripción de calificación
 - Valor medio tras la calificación
 - Cantidad de elementos: elementos que hacen match por los filtros enviados. Pueden ser más que el límite de la página.
 - Cantidad de páginas disponibles.
 - Número de página.

GET /perfil/sugerencias

- Respuesta esperada: 200 "Ok".
- Hace un muchos contra muchos de las repetidas de otros usuarios contra las faltantes del usuario en sesión dentro del Token, devolviendo repetidas y usuarios que permitan un intercambio satisfactorio.
- Errores posibles: 400 "Bad Request".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Página
 - Límite por página
 - Tipo de sugerencias: ("1a1" | "1aN" | "Na1" | "NaN")
- Devuelve:
 - Lista de contenido:
 - Perfil del usuario con el cual se matcheo la sugerencia.
 - Figuritas que al sugerido podría interesarles.
 - Figuritas que al perfil matcheado podría interesarles que el sugerido tenga repetida.
 - Cantidad de elementos: elementos que hacen match por los filtros enviados. Pueden ser más que el límite de la página.
 - Cantidad de páginas disponibles.
 - Número de página.

GET /perfil/contadores

- Respuesta esperada: 200 "Ok".
- Ruta destinada a realizar conteos de ciertos aspectos de un perfil, algunos pueden ser: Cantidad de repetidas, cantidad de faltantes, etc.
- Se requieren los campos:
 - Token (JsonWebToken)
- Devuelve:
 - Por cada contador:
 - Nombre del contador
 - Valor del contador

GET /perfil/notificaciones

- Respuesta esperada: 200 "Ok".
- Búsqueda de las notificaciones de un perfil dentro del Token.
- Se requieren los campos:
 - Token (JsonWebToken)
- Devuelve:
 - Por cada notificación:
 - Id de la notificación
 - Mensaje de la notificación
 - Perfil del usuario notificado

GET /perfil

- Respuesta esperada: 200 "Ok".
- Búsqueda del perfil guardado en el Token.
- Se requieren los campos:
 - Token (JsonWebToken)
- Devuelve:
 - Id el perfil.
 - Id del usuario del perfil
 - Nombre del perfil
 - Calificación media del perfil
 - Iniciales del perfil

Colecciones

POST /colecciones/faltantes

- Respuesta esperada: 201 "Created".
- Relaciona una figurita como faltante para el usuario almacenado en el Token.
- Errores posibles: 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Body:
 - Id de la figurita a relacionar como faltante.
- No devuelve información.

POST /colecciones/repetidas

- Respuesta esperada: 201 "Created".
- Relaciona una figurita como repetida, una cantidad de repetidas y métodos de intercambio para el usuario almacenado en el Token.
- Errores posibles: 400 "Bad Request", 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Body:
 - Id de la figurita a relacionar como faltante.
 - Cantidad de repeticiones.
 - Lista de métodos de intercambio
- No devuelve información.

GET /colecciones/faltantes

- Respuesta esperada: 200 "Ok".
- Realiza la búsqueda de las figuritas faltantes del usuario almacenadas en el Token.
- Errores posibles: 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Body:
 - Limite: Elementos por página
 - Página actual.
- Devuelve:
 - Lista de contenido:
 - Id de la figurita
 - Número de la figurita
 - Nombre del jugador
 - Selección del jugador
 - Cantidad de elementos: elementos que hacen match por los filtros enviados. Pueden ser más que el límite de la página.
 - Cantidad de páginas disponibles.
 - Número de página.

GET /colecciones/repetidas

- Respuesta esperada: 200 "Ok".
- Realiza la búsqueda de las figuritas repetidas, cantidad y método del usuario almacenadas en el Token.
- Errores posibles: 400 "Bad Request", 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Body:
 - Limite: Elementos por página
 - Página actual.
 - Método de intercambio buscado
- Devuelve:
 - Lista de contenido:
 - Id de la figurita

- Número de la figurita
- Nombre del jugador
- Selección
- Cantidad disponible de esa repetida
- Cantidad reservada de esa repetida
- Métodos de intercambio permitidos en esa repetida
- Id del usuario dueño de esa repetida
- Nombre del usuario dueño de esa repetida
- Calificación media del usuario dueño de esa repetida
- Posición del jugador
- Cantidad de elementos: elementos que hacen match por los filtros enviados. Pueden ser más que el límite de la página.
- Cantidad de páginas disponibles.
- Número de página.

Figuritas

GET /figuritas

- Respuesta esperada: 200 "Ok".
- Realiza búsqueda de las figuritas repetidas de otros usuarios persistidas en el sistema.
- Filtros admitidos:
 - Query: String destinado a ser utilizado como fulltext search. Si este campo está presente, los demás no se tendrán en cuenta.
 - Número de figuritas.
 - Selección de equipo.
 - Nombre del jugador.
 - Tipo de método de intercambio.
 - Página.
 - Límite: Elementos por página.
- Devuelve:
 - Lista de contenido:
 - Id de la figurita
 - Número de la figurita
 - Nombre del jugador
 - Selección
 - Cantidad disponible de esa repetida
 - Cantidad reservada de esa repetida
 - Métodos de intercambio permitidos en esa repetida
 - Id del usuario dueño de esa repetida
 - Nombre del usuario dueño de esa repetida
 - Calificación media del usuario dueño de esa repetida
 - Posición del jugador
 - Cantidad de elementos: elementos que hacen match por los filtros enviados. Pueden ser más que el límite de la página.
 - Cantidad de páginas disponibles.
 - Número de página.

Propuestas

POST /propuestas

- Respuesta esperada: 201 "Created".
- Crea y persiste una propuesta en el sistema.
- Errores posibles: 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Body:
 - Id del usuario a proponer.
 - Id de la figurita que se quiere conseguir a través de la propuesta.
 - Lista de ids de figuritas que se ofrecen al destinatario.
- No devuelve información.

PATCH /propuestas/{id_propuesta}/aceptar

- Respuesta esperada: 204 "No Content".
- Siendo el usuario destinatario de la propuesta enviada por PathParam, acepta esa propuesta, realizando los cambios en la persistencia del sistema.
- Errores posibles: 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Id de la propuesta por PathParam
- No devuelve información.

PATCH /propuestas/{id_propuesta}/rechazar

- Respuesta esperada: 204 "No Content".
- Siendo el usuario destinatario de la propuesta enviada por PathParam, rechaza esa propuesta, cambiando el estado de la propuesta.
- Errores posibles: 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Id de la propuesta por PathParam
- No devuelve información.

PATCH /propuestas/{id_propuesta}/cancelar

- Respuesta esperada: 204 "No Content".
- Siendo el usuario autor de la propuesta enviada por PathParam, cancela la propuesta, cambiando el estado de la propuesta.
- Errores posibles: 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Id de la propuesta por PathParam
- No devuelve información.

GET /propuestas

- Respuesta esperada: 200 "Ok".

- Realiza búsqueda de las propuestas de intercambio persistidas en el sistema, pudiendo restringir el resultado mediante filtros.
- Filtros admitidos:
 - Tipo de query: (“ENVIADAS” | “RECIBIDAS”).
 - Estado de la propuesta: (“ACEPTADO” | “RECHAZADO” | “CANCELADO” | “PENDIENTE”).
 - Página.
 - Límite: Elementos por página.
- Devuelve:
 - Lista de contenido:
 - Id de la propuesta.
 - Autor de la propuesta.
 - Destinatario de la propuesta.
 - Figurita buscada por la propuesta.
 - Figuritas ofrecidas en la propuesta.
 - Estado actual de la propuesta.
 - Booleano que identifica si ya se calificó al autor de la propuesta.
 - Cantidad de elementos: elementos que hacen match por los filtros enviados. Pueden ser más que el límite de la página.
 - Cantidad de páginas disponibles.
 - Número de página.

Subastas

POST /subastas

- Respuesta esperada: 200 “Ok”.
- Crea y persiste una subasta en el sistema.
- Errores posibles: 400 “Bad Request”, 404 “Not Found”.
- Se requieren los campos:
 - Token (JsonWebToken)
 - Body:
 - Id de la figurita a subastar.
 - Duración de la subasta en horas.
 - Lista de ids de las figuritas que deben formar parte de las ofertas en la subasta.
 - Valor mínimo de calificación media que debe tener un perfil para poder ofrecer en la subasta.
- No devuelve información.

POST /subastas/{sub_id}/ofertas

- Respuesta esperada: 200 “Ok”.
- Agrega una oferta en la subasta enviada por PathParam. La oferta debe cumplir con las condiciones mínimas de la subasta para ser persistida.
- Errores posibles: 400 “Bad Request”, 404 “Not Found”.
- Se requieren los campos:
 - Token (JsonWebToken)
 - Id de la subasta en PathParam

- Body:
 - Lista de ids de figuritas que se ofrecen al autor de la subasta.
- No devuelve información.

PATCH /subastas/{sub_id}/ofertas/{oferta_id}

- Respuesta esperada: 200 "Ok".
- Siendo el autor de la propuesta (oferta_id) y un ofertador previo de la subasta (sub_id), permite reescribir los contenidos de la oferta en la subasta.
- Errores posibles: 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Id de la oferta por PathParam
 - Id de la subasta en PathParam
 - Body:
 - Lista de ids de figuritas que reemplazará a la oferta anterior.
- No devuelve información.

PATCH /subastas/{sub_id}/ofertas/{oferta_id}/cancelar

- Respuesta esperada: 200 "Ok".
- Siendo el autor de la propuesta (oferta_id) y un ofertador previo de la subasta (sub_id), cambia el estado de la oferta a "CANCELADO".
- Errores posibles: 400 "Bad Request", 401 "Unauthorized", 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Id de la oferta por PathParam
 - Id de la subasta en PathParam
- No devuelve información.

PATCH /subastas/{sub_id}/ofertas/{oferta_id}/seleccionar

- Respuesta esperada: 200 "Ok".
- Siendo el autor de la propuesta (oferta_id) y un ofertador previo de la subasta (sub_id), cambia el estado de la oferta a "SELECCIONADO". Marca al resto de usuarios que esa oferta está siendo la ganadora y, si el tiempo termina o el dueño de la subasta la finaliza, es la seleccionada como ganadora.
- Errores posibles: 400 "Bad Request", 401 "Unauthorized", 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Id de la oferta por PathParam
 - Id de la subasta en PathParam
- No devuelve información.

PATCH /subastas/{sub_id}/ofertas/{oferta_id}/rechazar

- Respuesta esperada: 200 "Ok".
- Siendo el autor de la propuesta (oferta_id) y un ofertador previo de la subasta (sub_id), cambia el estado de la oferta a "RECHAZADO". Le indica al oferente que, si quiere ganar, va a tener que mejorar la oferta.

- Errores posibles: 400 "Bad Request", 401 "Unauthorized", 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Id de la oferta por PathParam
 - Id de la subasta en PathParam
- No devuelve información.

PATCH /subastas/{sub_id}/cancelar

- Respuesta esperada: 200 "Ok".
- Siendo el autor de la subasta (sub_id), provoca que la subasta finalice sin definir un ganador. Esta acción solo altera a la subasta, las ofertas presentadas no serán tenidas en cuenta.
- Errores posibles: 400 "Bad Request", 401 "Unauthorized", 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Id de la subasta en PathParam
- No devuelve información.

PATCH /subastas/{sub_id}/cerrar

- Respuesta esperada: 200 "Ok".
- Siendo el autor de la subasta (sub_id), provoca que la subasta finalice definiendo al ganador seleccionado. Esta acción altera la subasta y a la oferta seleccionada.
- Errores posibles: 400 "Bad Request", 401 "Unauthorized", 404 "Not Found".
- Se requieren los campos:
 - Token (JsonWebToken)
 - Id de la subasta en PathParam
- No devuelve información.

GET /subastas

- Respuesta esperada: 200 "Ok".
- Realiza búsqueda de las subastas persistidas en el sistema.
- Filtros admitidos:
 - Tipo de query: ("ENVIADAS" | "RECIBIDAS").
 - Estado de la propuesta: ("ACEPTADO" | "RECHAZADO" | "CANCELADO" | "PENDIENTE").
 - Página.
 - Límite: Elementos por página.
- Devuelve:
 - Lista de contenido:
 - Id de la subasta.
 - Autor de la subasta.
 - Fecha de inicio de la subasta.
 - Fecha de cierre de la subasta.
 - Tiempo restante de la subasta hasta que cierre automáticamente.
 - Figurita subastada.
 - Figuritas solicitadas como condición mínima.
 - Calificación mínima solicitada.

- Ofertas anteriores en la subasta.
- Cantidad de elementos: elementos que hacen match por los filtros enviados. Pueden ser más que el límite de la página.
- Cantidad de páginas disponibles.
- Número de página.

1.4 Persistencia

Para la persistencia se decidió trabajar con **MongoDB**, ya que:

- Nos permite tener dinamismo en el manejo de la información.
- Conservar relaciones y consultas flexibles.
- El equipo tiene un mayor conocimiento en el trabajo con bases de datos documentales.
- Los frameworks seleccionados nos brindan robustez y son ampliamente conocidos en la industria.
- El mapeo de Documental y objetos es semi-directo.

Paginación

Se desarrolló una clase **PaginaResultado<>**, con el objetivo de devolver listas de objetos con atributos comunes como:

- **Data**: Lista de objetos
- **Numero**: Número de página actual
- **CantidadDeElementos**: Resultados totales que cumplen con una query.
- **CantidadDePaginas**: Cantidad de páginas que entran en esa cantidad de elementos, utilizado para saber cuántas páginas quedan hasta llegar al fin de la lista.

Si se desea devolver más información que la mencionada arriba, se pueden componer clases entre ellas, permitiendo un uso común entre ellas.

Objetos Embebidos

En algunas clases determinamos que, para aprovechar mejor el motor de mongoDB, la mejor elección era embeber información dentro de la colección (en lugar de referenciar y/o hacerla una colección aparte). Esto se dio en dos clases particularmente:

- **Subasta.ofertas**: Las ofertas (lista de propuestas) se las embebe dentro de la colección subasta, ya que las ofertas de una subasta solo existen para la subasta en sí y no tienen sentido fuera de ella.
- **Colección.repetidas**: Las repetidas (**FiguritaIntercambiable**) de un perfil, son contenidas en el objeto de colección, y nuevamente solo tienen sentido dentro de ella. Lo que significa que, ninguna **FiguritaIntercambiable** existe fuera de una y solo una colección.

Listas Lazy

Se desarrolló un comportamiento extra en algunas de las clases que implementan listas en sus atributos, diseñada para evitar la carga de elementos cuando algunas acciones no interactúan con ellos. Esto se implementó de dos maneras distintas:

- **Lazy nativo**: Implementado por el framework Spring Data MongoDB.

- **Lazy manual:** En algunos casos, donde las listas son embebidas en la colección, no podemos usar el lazy que proviene del **Tag** (ya que embeber se denota sin tag). Para implementar lazy en estos casos, se decidió crear clases que tengan el objetivo de decirle a la query que campos cargar de ese objeto.

Otras consideraciones

A la hora de desarrollar el sistema de calificación, nos percatamos de que el cálculo del promedio en memoria podría ocasionar problemas en el futuro. Es por ello que se decidió guardar un valor medio y una cantidad de calificaciones asociada a ese perfil.

De esta manera no es necesario cargar todas las calificaciones en memoria (las cuales pueden ser muchas), permitiéndonos una mejor eficiencia de recursos (memoria y tiempo) a cambio de espacio en disco.

2. Frontend

2.1 Dependencias

Frameworks

React 19

Framework principal utilizado para el desarrollo de la interfaz de usuario de la aplicación. Nos permitió construir una SPA (Single Page Application) basada en componentes reutilizables, facilitando la modularización, mantenimiento y escalabilidad del frontend. Además, su ecosistema y compatibilidad con múltiples librerías simplificaron la integración de herramientas complementarias.

Librerías

React Router 7

Librería utilizada para la gestión de rutas dentro de la aplicación. Permitió implementar navegación dinámica entre vistas sin necesidad de recargar la página, mejorando la experiencia del usuario y facilitando la organización de las distintas pantallas del sistema.

Axios

Librería altamente conocida por el equipo, que nos permitió validar errores de peticiones a la API, entre otras cosas.

Bootstrap

Librería de estilos para nuestros componentes de React, en conjunto con estilos configurados por el equipo. Se decidió utilizarla por su facilidad para integrar estilos predeterminados y consistentes entre componentes, manteniendo la aplicación responsive y acelerando el desarrollo visual de la interfaz.

Herramientas y otros

Vite 8

Herramienta de build y entorno de desarrollo utilizada para el frontend. Fue elegida por su rapidez en tiempos de compilación y recarga en caliente (Hot Reload), permitiendo un flujo de desarrollo más ágil y eficiente en comparación con otras alternativas tradicionales.

Nginx

Servidor web utilizado para servir la aplicación frontend en ambientes de producción. Permitió desplegar la aplicación de manera eficiente, manejando archivos estáticos y facilitando la configuración del entorno de despliegue.

3. Infraestructura

3.1 Docker

Para la containerización se decidió trabajar con multi-stage builds tanto en el backend como en el frontend. La imagen final no tiene el compilador ni las dependencias de build, únicamente el ejecutable. Además, cada dockerfile define stages separadas para desarrollo y producción.

3.2 Backend

El Dockerfile del backend define cuatro stages:

- **dependencies:** Parte de maven:3.9-eclipse-temurin-17 y descarga todas las dependencias y plugins con mvn dependency:go-offline. Al separar este paso, Docker cachea la capa de dependencias y evita redescargarlas en cada build mientras el pom.xml no cambie.
- **dev:** Agrega el código fuente sobre la etapa anterior, sin correr tests ni validaciones. Es el stage objetivo para el entorno de desarrollo.

- **build:** Corre mvn verify sobre la etapa dev, ejecutando tests, análisis de Spot Bugs y cobertura mínima con JaCoCo. En CI se puede sustituir por mvn package -DskipTests si los tests ya corrieron en un step previo, vía el argumento MAVEN_ARGS.
- **runtime:** Parte de eclipse-temurin:17-jre-alpine (JRE Alpine, sin JDK ni Maven). Crea un usuario sin privilegios (appuser) y copia únicamente el JAR generado. La plataforma se fuerza a linux/amd64 porque la imagen Alpine de Eclipse Temurin no tiene soporte ARM64 . Es la imagen final utilizada en producción.

3.2 Frontend

El Dockerfile del frontend define cuatro stages:

- **deps** Parte de node:22-alpine y corre npm ci para instalar los paquetes. Al separar este paso, Docker cachea node_modules mientras package-lock.json no cambie.
- **dev:** Agrega el código fuente sobre el stage anterior sin compilar. Es el stage objetivo para el entorno de desarrollo.
- **builder:** Corre npm run build sobre el stage dev, generando los assets estáticos en /app/dist.
- **production:** Parte de nginxinc/nginx-unprivileged:alpine, que corre como usuario no root en el puerto 8080. Copia el directorio /dist generado por la etapa anterior y la configuración personalizada de Nginx. Es la imagen final utilizada en producción

3.3 Compose prod

El archivo docker-compose.yml orquesta dos servicios: backend y frontend. Para garantizar que las primeras llamadas a la API no fallen durante el arranque, el frontend declara depends_on con condition: service_healthy sobre el backend. El backend expone /ping como endpoint de salud, con el siguiente health check:

- Intervalo: 10s
- Timeout: 5s
- Reintentos: 6
- Período inicial: 15s (margen para que Spring Boot inicialice)

Ambos servicios tienen restart: unless-stopped para que Docker los recupere automáticamente ante una caída.

3.3 Compose dev

El archivo `docker-compose.dev.yml` incorpora un servicio adicional de MongoDB (`mongo:7`) con réplica set, necesario para soportar transacciones. El proceso de inicialización del replica set se delega a un contenedor efímero (`mongo-init`) que corre `rs.initiate()` si el replica set aún no fue configurado, ignorando la operación en arranques posteriores.

El backend buildea hasta el stage dev y corre con `mvn spring-boot:run`. El frontend buildea hasta el stage dev y corre con `npm run dev --host`, habilitando el servidor de Vite en modo red.

Los cambios en el código fuente del frontend se sincronizan al contenedor mediante Compose Watch, que transfiere archivos a través del socket Docker sin requerir volume mounts. Vite detecta los cambios y actualiza el browser automáticamente (Hot Reload). Los cambios en el backend requieren un redespigie manual con `make dev-back`.

4. Uso de la Inteligencia Artificial

En todos los casos se trabajó con prompts detallados, proporcionando contexto específico del sistema (estructura del proyecto, casos de uso, tecnologías utilizadas) para obtener resultados más precisos y reducir la necesidad de correcciones.

4.1 Creación de test según casos de uso

Se usó IA para generar casos de prueba a partir de los casos de uso del sistema. Los tests cubrieron las tres capas de la aplicación: repositorios, servicios y controladores. En los controladores se validaron los endpoints de la API REST (respuestas esperadas, códigos HTTP y manejo de errores), en los servicios se probó la lógica de negocio del sistema de figuritas (como la verificación de duplicados, los intercambios entre usuarios y la completitud del álbum), y en los repositorios se verificó el acceso y la persistencia correcta de los datos. La IA ayudó a identificar escenarios límite y a escribir los tests de forma más ágil.

4.2 Diseño de las plantillas para el frontend

A partir de la descripción de las funcionalidades del sistema (User Stories), se generaron propuestas de layouts y estructuras de componentes que luego fueron adaptadas y refinadas por el equipo.

4.3 Generación de Componentes React

Durante el desarrollo del frontend se utilizó IA para generar componentes React reutilizables. Los componentes generados incluyen tanto elementos de UI genéricos (secciones, botones, modales) como componentes específicos del dominio del sistema, como la carta de figurita, la grilla del álbum y las vistas de intercambio. En todos los casos,

el código generado fue revisado, adaptado e integrado al proyecto por el equipo, asegurando coherencia con la arquitectura y los estándares definidos.

4.4 Generación de Documentación

Por un lado, se generaron los JavaDocs de las clases del backend, documentando métodos, parámetros y valores de retorno a partir del código existente. Por otro lado, se usó para redactar documentación del proyecto, en apartados como:

- Infraestructura.

4.5 Debugging errores del Front y Back

Ante fallos o comportamientos inesperados, se compartía el mensaje de error junto con el fragmento de código relevante y contexto del sistema, y la IA ayudaba a identificar la causa y sugerir posibles soluciones.

4.6 Revisión de Pull Request en Github

Antes de mergear cambios, se compartía el diff para que la IA detectara posibles problemas, inconsistencias con el resto del sistema o mejoras en la implementación.